Test Automation Framework (TAF::Perl)

TAF Profiler Quick-Start

MariaDB Foundation https://mariadb.org

Open, vendor-neutral, community-driven tooling for database testing and benchmarking.

TAF-Perl is maintained and published by the MariaDB Foundation to support transparent, reproducible, and fair performance evaluation across the open source database ecosystem. The framework is part of the Foundation's mission to provide open tooling that benefits the entire database community.

TAF Profiling Quick Start Guide

1. What profiling is in TAF

Profiling in TAF lets you capture performance data during a run without changing your workload or test suite. When profiling is enabled, TAF loads a profiler module (for example, taf-perf) and controls when it starts and stops.

You use profiling when you want to understand where time is spent inside the database or client under load, not just how many transactions per second you get.
Profiling is disabled by default. Nothing happens unless you explicitly turn it on.

2. When to use profiling

Typical situations where profiling is useful:

- Comparing two database versions (for example, 11 vs 12) and you see a gap.
- Investigating a regression that only appears at higher thread counts.
- Understanding steady-state behavior after warmup.
- Capturing CPU hotspots during a specific phase of a test.

You generally do not run profiling on every test. It adds overhead and produces extra output. Think of it as a targeted tool you enable when you have a question you want to answer.

3. Basic workflow

A typical profiling workflow in TAF looks like this:

1. Enable profiling for the run.
2. Choose which profiler module to use (default is taf-perf).
3. Decide when profiling should start (start delay).
4. Decide how long profiling should run (duration) or whether it should run continuously.
5. Optionally request a report or flamegraph data.

TAF then:

- Loads the profiler module.
- Waits for the start delay.
- Starts profiling.
- Stops profiling after the duration, or when the module decides to stop.
- Writes profiler output into the run directory.

4. Common usage patterns

4.1 Steady-state profiling after warmup

Goal: avoid startup noise and capture a clean steady-state window.

Example:

```
--profiler-enabled
--profiler-start-delay=120
--profiler-duration=30
```

This means:

- Run the workload normally for 120 seconds.
- Then profile for 30 seconds.
- Then stop profiling and let the iteration finish.

This is a good pattern for comparing two versions under the same conditions.

4.2 Continuous profiling for the whole iteration

Goal: see everything that happens during the iteration.

Example:

```
--profiler-enabled
--profiler-continuous
```

This means:

- Start profiling when the iteration begins.
- Keep profiling until the iteration ends or the profiler module stops it.

This is useful for exploratory work or long-running tests, but it can generate a lot of data.

4.3 Custom profiler options

Goal: control what the profiler collects.

Example (taf-perf):

```
--profiler-enabled
--profiler-opts="-e cycles,instructions,branches"
```

The string passed to --profiler-opts is interpreted by the profiler module. For taf-perf, it is passed directly to perf as its event options.

5. Options and what they mean

Below is a description of the profiling-related options and properties. The command-line option (O) overrides the matching property (P) if both are set.

Enable or disable profiling:

O: --profiler-enabled P: taf.profiler_enabled=<bool>

When true, TAF loads the selected profiler module and manages it for the run. When false, no profiling occurs. Default is false.

Select the profiler module:

O: --profiler-lib=<name> P: taf.profiler_lib=<name>

This chooses which profiler implementation to load. Example: taf-perf. The name must match a known profiler module. Default is taf-perf.

Control when profiling starts:

O: --profiler-start-delay=<seconds> P: taf.profiler_start_delay=<seconds>

This is the delay from the start of the iteration to the moment profiling begins. Use this to skip warmup or startup noise. Default is 0.

Control how long profiling runs (duration mode):

O: --profiler-duration=<seconds> P: taf.profiler_duration=<seconds>

This is how long profiling runs once it starts, when not in continuous mode. The sum of start delay and duration must fit within the iteration duration. Default is 0 (no duration-based profiling unless continuous is used).

Enable continuous profiling:

O: --profiler-continuous P: taf.profiler_continuous=<bool>

When true, the profiler runs continuously until the profiler module stops it or the iteration ends. When false, duration mode is used if a duration is set. Default is false.

Pass profiler-specific options:

O: --profiler-opts="<string>" P: taf.profiler_opts="<string>"

This string is passed to the profiler module. For taf-perf, it is passed directly to perf as its event options.

Default is:

-e cpu-clock:pp

Control report generation:

O: --profiler-generate-report P: taf.profiler_generate_report=<bool>

When true, the profiler module generates a human-readable report after profiling. The exact format depends on the module. Default is false.

Control flamegraph data generation:

O: --profiler-generate-flamegraph P: taf.profiler_generate_flamegraph=<bool>

When true, the profiler module generates raw stack-trace data suitable for external flamegraph tools. The exact output depends on the module. Default is false.

7. Automatic Flamegraph Generation

TAF can automatically generate flamegraph data when profiling is enabled. No external tools or manual steps are required. When flamegraph generation is turned on, TAF:

1. Captures stack traces using the selected profiler module
2. Converts them into folded stack format
3. Produces a ready-to-view SVG flamegraph
4. Stores all artifacts inside the iteration's result directory and archive

This ensures reproducible, portable, and consistent profiling output across environments.

To enable flamegraph generation:

Command-line:

--profiler-enabled --profiler-generate-flamegraph

Properties file:

taf.profiler_enabled=true taf.profiler_generate_flamegraph=true

TAF will produce the following files:

- perf.data
- perf_report.txt
- flamegraph.txt
- flamegraph.txt.folded
- flamegraph.txt.svg

These files are included in the archived results for long-term analysis and regression tracking.

TAF's flamegraph output is produced using the FlameGraph scripts authored by Brendan Gregg. The toolkit is included as-is and executed automatically to convert perf stack traces into folded data and SVG flamegraphs.

8. Best practices

- Always allow some warmup before profiling if you care about steady-state.
- Use start delay + duration for targeted windows instead of profiling everything by default.

- Keep profiler options simple unless you have a specific question that requires detailed events.
- Store profiler output alongside your benchmark results so you can revisit regressions later.
- Use the same profiling settings when comparing two versions so the results are comparable.

9. Summary

Profiling in TAF is a controlled way to capture performance data during a run. You turn it on when you have a question about where time is going, choose when it should run, and let TAF manage the profiler lifecycle for you.

End of Document

This Quick Start was prepared and published by the MariaDB Foundation https://mariadb.org

TAF::Perl is part of the Foundation's commitment to open, vendor-neutral tooling for the database ecosystem. The framework is maintained to support fair benchmarking, reproducible testing, and transparent engineering practices across all database makers.

For contributions, feedback, or questions, please visit the Foundation website or contact the maintainers directly.